

# Project Assignment

TypeORM

# Outline

- What's TypeORM?
- DataSource
- Entities definition
- Entities relationships
- Repositories
- ORM testing

# What's TypeORM?

# TypeORM

TypeORM is an Object-Relational Mapping (ORM) library for TypeScript and JavaScript. It allows developers to work with relational databases using an object-oriented approach, eliminating the need to write raw SQL queries.

TypeORM supports multiple database engines and uses decorators to define entities definition.

It also provides migration support, query building, and relations management, making database interactions easier and more structured.

# Key Features

- Fully written in TypeScript (but supports JavaScript)
- Uses decorators to define entities and relationships
- Supports multiple database engines
- Includes automatic migrations and schema synchronization
- Allows lazy and eager loading for relations
- Compatible with NestJS, Express, and other frameworks

# DataSource

- In TypeORM, a DataSource is the core object that manages the connection to the database
- It defines database settings, entity locations, and synchronization options
- TypeORM requires a single DataSource instance to interact with the database
- A DataSource must be initialized before executing any database operations
- A DataSource must be destroyed at the end of the application lifecycle
- A DataSource object must be created in a separate file (database/connection.ts)

# DataSource connection

- The connection to the DataSource needs to be configured defining
  - database type (SQLite, MySQL...)
  - database name or location access credentials (if needed)
  - entities location
  - schema synchronization
  - logging
- synchronize
  - if true, it automatically creates and updates database tables based on entity definitions
  - useful for development, but dangerous in production as it can modify or delete data

# SQLite

- Lightweight, file-based relational database that does not require a separate server
- Used for embedded applications, prototyping, and testing
- Fully supported by TypeORM for file-based and in-memory databases
- SQLite Operating Modes
  - File-based: stores data in a .sqlite file, persisting between sessions
    - `database: "database.sqlite"`
  - In-memory: runs entirely in RAM, losing all data when the app stops
    - `database: ":memory:"`
    - Useful for testing and temporary data storage

# Entities

- @Entity() defines tables
- @Column() defines table fields
  - nullable: whether a field can be NULL, default value = false
  - unique: enforces uniqueness on a column, default value = false
  - default: sets a default value for the column
- @PrimaryKeyColumn() defines the primary key field of an entity
  - generated: defines key generation strategy
    - supported strategies: "increment" | "uuid" | "rowid" | "identity"
  - type: specifies column type for custom primary keys (e.g., "uuid", "varchar")

# Relationships - Decorator

- All relationships in TypeORM are defined using the same decorator structure:
- `@RelationshipDecorator(() => TargetEntity, (target) => target.property, options)`
- Parameters:
  - TargetEntity: the class of the related entity
  - target.property: the property in the related entity that references this entity (only needed in bidirectional relations)
  - options: defines how the relationship behaves (cascade, deletion rules, loading strategy etc.)

# Relationships - Common options

- Cascade Options (cascade: true | ["insert", "update", "remove"])
  - true: applies to insert, update, and delete
  - "insert": saves related entities automatically when the parent is saved
  - "update": propagates updates to related entities when the parent is updated
  - "remove": deletes related entities when the parent is deleted
  - Handled by TypeORM in memory before committing changes to the database
- Delete Behavior (onDelete)
  - "CASCADE": deletes related records automatically when the parent is deleted
  - "SET NULL": sets the foreign key to NULL instead of deleting related records
  - "RESTRICT": prevents deletion if related records exist
  - Handled by the database itself, enforcing constraints at the SQL level

# Relationships - Common options

- Loading Strategy

- eager: true

- The related entity is loaded automatically when the parent entity is loaded
    - The property must have type Entity
    - Cannot be used on both sides of a bidirectional relation

- lazy: true

- Lazy loading is the default if eager is not specified
    - The property must have type Promise<Entity>
    - The related entity is fetched only when accessed, e.g. `let author = await post.author;`

# OneToOne

- Decorator: @OneToOne()
- Requires @JoinColumn() on the owning side to define the foreign key
- Supported options:
  - cascade: true | ["insert", "update", "remove"]
  - onDelete: "CASCADE" | "SET NULL" | "RESTRICT"

# OneToMany

- Decorator: `@OneToMany()`
- Inverse side of `@ManyToOne`.
- No `@JoinColumn()` is needed, as the foreign key is in the other table
- `@OneToMany` is just a virtual mapping for easier querying
- For the latter reason, it doesn't support specific relationship option

# ManyToOne

- Decorator: @ManyToOne()
- The foreign key is always stored in the entity with @ManyToOne
- Requires @JoinColumn() to specify the foreign key
- Supported options:
  - cascade: true | ["insert", "update", "remove"]
  - onDelete: "CASCADE" | "SET NULL" | "RESTRICT"
  - nullable: true/false (default: false)

# ManyToMany

- Decorator: @ManyToMany()
- Requires @JoinTable() on the owning side
- TypeORM automatically creates a junction table to manage relationships
- Supported options:
  - cascade: true | ["insert", "update", "remove"]
  - No onDelete support (handled via constraints on the junction table)

# Bidirectional relationships

- TypeORM does not support eager loading on both sides of a bidirectional relationship
- If both sides have eager: true, TypeORM would enter an infinite loop while trying to load related entities
  - Set eager: true only on one side of the relationship
- When a relation is lazy, accessing the property does not return the object directly, but a Promise
- The actual entity is fetched only when the property is accessed

# Repositories

- Repositories provide high-level access to database operations for a specific entity
- TypeORM provides a built-in implementation of the Repository pattern for each entity
- Retrieved via `DataSource.getRepository(Entity)`
- Common methods:
  - `find`, `findOne`, `save`, `remove`, `delete`
- Filtering is done via TypeScript objects, not SQL:
  - Conditions are passed using entity property names
  - Sorting is handled with the `order` option
- **You can work entirely in TypeScript, without writing any SQL**

# Custom Repositories

- Define your own class to wrap and extend repository logic.
- Useful to centralize domain-specific queries.
- Internally use `DataSource.getRepository(Entity)` to access the base repository.
- Keeps business logic separated from direct database access.
- Encourages reuse and testability of complex queries.

# ORM Testing

- Tests must not touch the real application database to avoid altering persistent data
- First approach is mocking lower persistence layer: allows fast, isolated tests without touching the database
- In some cases (e.g. end-to-end tests), mocking is not possible because the implementation is unknown or external
  - In those cases, use a separate test database, ideally an in-memory SQLite DataSource
- **Never use the application's real database during testing**
- Always ensure tests run on an isolated and disposable DataSource

# ORM Testing

- Entities are pure data holders, they contain no logic and do not need to be tested directly
- Logic is located in repositories and services, and these are the primary targets for testing
- When using an ORM, avoid testing the ORM itself—focus on your application logic
- Two main strategies:
  - Mocking the repository (unit testing)
  - Test DataSource (realistic but heavier)

# Mocking the Persistence Layer

- The recommended approach for unit testing business logic
- Replace TypeORM's native repository methods with Jest mocks
  - Example: mock find, findOne, save, remove, delete methods
- Useful for isolating code and testing logic without persistence
- Avoids the need to configure or initialize any real database

# Test DataSource

- Suitable for end-to-end tests
- Use a real database: SQLite in memory
- Database is recreated from scratch on each run
- Closer to real behavior, but slower than mocks
- Requires full DataSource management: initialization, cleanup and release